

Animating React Apps

Easy Guide

Contents

I	Basic Animations with CSS	2
1	Animating with CSS Transitions	3
1.1	What are CSS Transitions?	3
1.1.1	Basic Example	3
1.2	Transition Properties	3
1.3	Common Transition Properties	3
1.4	Timing Functions	4
1.5	React Example	4
2	Animating with CSS Animations	5
2.1	What are CSS Animations?	5
2.2	Basic Keyframes	5
2.3	Animation Properties	5
2.4	Common Animation Properties	6
2.5	Multiple Keyframe Steps	6
2.6	React Component with Animation	6
II	Advanced Animations with Framer Motion	8
3	Introducing Framer Motion	9
3.1	What is Framer Motion?	9
3.2	Installation	9
3.3	Why Use Framer Motion?	9
3.4	Basic Example	9
3.5	Motion Components	10
3.6	Three Key Props	10
4	Framer Motion Basics & Fundamentals	11
4.1	Initial and Animate	11
4.2	Exit Animations	11
4.3	Transform Properties	12
4.4	Transition Types	12
5	Animating Between Conditional Values	14
5.1	Conditional Animation	14
5.2	Complex Conditional Animation	14

6	Adding Entry Animations	16
6.1	Entry Animation Pattern	16
6.2	Different Entry Animations	16
6.2.1	Fade In	16
6.2.2	Slide In from Left	16
6.2.3	Scale Up	16
6.2.4	Rotate In	17
6.3	List Entry Animation	17
7	Animating Element Disappearances / Removal	18
7.1	Exit Animation with AnimatePresence	18
7.2	List Item Removal	19
8	Making Elements “Pop” With Hover Animations	20
8.1	Hover Animation	20
8.2	Hover Props	20
8.3	Complex Hover Animation	20
8.4	Icon Pop Effect	21
9	Reusing Animation States with Variants	22
9.1	What are Variants?	22
9.2	Reusable Variants	22
9.3	Variants with Transitions	23
10	Nested Animations & Variants	24
10.1	Parent-Child Animation	24
10.2	How Stagger Works	25
10.3	Delaying Parent Animation	25
11	Animating Staggered Lists	26
11.1	List Animation Pattern	26
11.2	Different Stagger Directions	27
12	Animating Colors & Working with Keyframes	28
12.1	Color Animation	28
12.2	Keyframes Array	28
12.3	Timing for Each Keyframe	29
12.4	Complex Color Animation	29
13	Imperative Animations	30
13.1	Using useAnimation Hook	30
13.2	Button Triggered Animation	30
13.3	Sequence of Animations	31
14	Animating Layout Changes	32
14.1	layoutId for Shared Layout Animation	32
14.2	Animated Flex Layout	32

15 Orchestrating Multi-Element Animations	34
15.1 Coordinating Multiple Elements	34
15.2 Sequential Multi-Element Animation	34
15.3 Staggered Multi-Element	35
16 Combining Animations With Layout Animations	36
16.1 Layout + Motion Together	36
17 Animating Shared Elements	38
17.1 Shared Element Transition	38
18 Re-triggering Animations via Keys	39
18.1 Using Keys to Reset Animations	39
18.2 Forcing Re-trigger	39
19 Scroll-based Animations	41
19.1 Using useViewportScroll	41
19.2 Scroll-based Opacity	41
19.3 Multiple Scroll Transforms	41

Part I

Basic Animations with CSS

Chapter 1

Animating with CSS Transitions

1.1 What are CSS Transitions?

Smooth changes from one state to another over time.

1.1.1 Basic Example

```
.button {  
  background-color: blue;  
  transition: background-color 0.3s ease;  
}  
  
.button:hover {  
  background-color: red;  
}  
  
/* Smoothly changes color over 0.3 seconds */
```

1.2 Transition Properties

`transition: property duration timing-function delay;`

Example:

```
transition: background-color 0.3s ease-in-out 0.1s;  
            ~~~~~ ~~~~~ ~~~~~ ~~~~~  
            what to animate  time  how to move delay
```

1.3 Common Transition Properties

- `opacity` - Fade in/out
- `transform` - Move, scale, rotate
- `color` - Change text color
- `background-color` - Change background

- width, height - Resize

1.4 Timing Functions

Function	Effect
linear	Same speed throughout
ease	Slow start/end, fast middle
ease-in	Slow start
ease-out	Slow end
ease-in-out	Slow start and end

1.5 React Example

```
import styles from './Button.module.css';
```

```
export default function Button() {  
  return (  
    <button className={styles.button}>  
      Hover me!  
    </button>  
  );  
}
```

```
/* Button.module.css */  
.button {  
  background-color: #3498db;  
  color: white;  
  padding: 10px 20px;  
  border: none;  
  cursor: pointer;  
  transition: all 0.3s ease;  
}
```

```
.button:hover {  
  background-color: #2980b9;  
  transform: scale(1.05);  
}
```

```
.button:active {  
  transform: scale(0.95);  
}
```

Perfect for: Hover effects, button states, simple color changes!

Chapter 2

Animating with CSS Animations

2.1 What are CSS Animations?

Keyframes define animation steps. More complex than transitions.

2.2 Basic Keyframes

```
@keyframes slideIn {
  0% {
    opacity: 0;
    transform: translateX(-100%);
  }

  100% {
    opacity: 1;
    transform: translateX(0);
  }
}

.element {
  animation: slideIn 0.5s ease-out;
}
```

2.3 Animation Properties

```
animation: name duration timing-function
           delay iteration-count direction;
```

Example:

```
animation: slideIn 0.5s ease-out 0.2s 1 normal;
           ~~~~~ ~~~~~ ~~~~~ ~~~~~ ~~~~~ ~~~~~
           name    time  easing    delay repeats direction
```


2.4 Common Animation Properties

- `animation-duration` - How long
- `animation-timing-function` - Speed curve
- `animation-delay` - Wait before starting
- `animation-iteration-count` - infinite or number
- `animation-direction` - normal, reverse, alternate

2.5 Multiple Keyframe Steps

```
@keyframes bounce {
  0% {
    transform: translateY(0);
  }

  50% {
    transform: translateY(-20px);
  }

  100% {
    transform: translateY(0);
  }
}

.bouncing {
  animation: bounce 1s ease-in-out infinite;
}
```

2.6 React Component with Animation

```
import styles from './Loading.module.css';

export default function Loading() {
  return (
    <div className={styles.spinner}>
      <div className={styles.dot}></div>
      <div className={styles.dot}></div>
      <div className={styles.dot}></div>
    </div>
  );
}

/* Loading.module.css */
@keyframes spin {
  0% { transform: rotate(0deg); }
```

```
    100% { transform: rotate(360deg); }
  }

.spinner {
  width: 50px;
  height: 50px;
  animation: spin 1s linear infinite;
}

.dot {
  position: absolute;
  width: 10px;
  height: 10px;
  background-color: #3498db;
  border-radius: 50%;
  animation: bounce 1.4s infinite ease-in-out both;
}
```

Perfect for: Loading spinners, complex sequences, repeated animations!

Part II

Advanced Animations with Framer Motion

Chapter 3

Introducing Framer Motion

3.1 What is Framer Motion?

JavaScript library that makes complex animations easy.

3.2 Installation

```
npm install framer-motion
```

3.3 Why Use Framer Motion?

1. **Simple API** - Less code than CSS
2. **Powerful features** - Gestures, layout animations
3. **Performance** - Optimized for 60fps
4. **Physics** - Spring animations, momentum
5. **Variants** - Reusable animation patterns

3.4 Basic Example

```
import { motion } from 'framer-motion';

export default function Box() {
  return (
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      transition={{ duration: 0.5 }}
    >
      Hello!
    </motion.div>
  );
}
```

3.5 Motion Components

Framer Motion provides motion versions of HTML elements:

- `motion.div`
- `motion.button`
- `motion.p`
- `motion.img`
- Any HTML element with `motion`

3.6 Three Key Props

- **initial** - Starting state
- **animate** - Target state
- **transition** - How to get there

Key Advantage: All animation logic in JavaScript, not CSS!

Chapter 4

Framer Motion Basics & Fundamentals

4.1 Initial and Animate

```
import { motion } from 'framer-motion';

export default function Card() {
  return (
    <motion.div
      initial={{ opacity: 0, y: -20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.6 }}
    >
      <p>Card content</p>
    </motion.div>
  );
}
```

When component mounts:

1. Set to `initial` state (invisible, 20px up)
2. Animate to `animate` state (visible, in place)
3. Takes 0.6 seconds

4.2 Exit Animations

```
import { motion, AnimatePresence } from 'framer-motion';

export default function Modal({ isOpen }) {
  return (
    <AnimatePresence>
      {isOpen && (
        <motion.div
```

```

        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        exit={{ opacity: 0 }}
        transition={{ duration: 0.3 }}
      >
        Modal content
      </motion.div>
    )}
  </AnimatePresence>
);
}

```

AnimatePresence lets components animate out before unmounting.

4.3 Transform Properties

```

// Position
x: 100           // Move 100px right
y: -50           // Move 50px up

// Scale
scale: 1.2       // Grow 20%
scaleX: 0.8      // Shrink width
scaleY: 1.5      // Grow height

// Rotation
rotate: 45       // Rotate 45 degrees

// Opacity
opacity: 0.5     // 50% transparent

// Skew
skewX: 10        // Skew X axis
skewY: 5         // Skew Y axis

```

4.4 Transition Types

```

// Tween (default)
transition={{ duration: 0.5, ease: "easeOut" }}

// Spring
transition={{
  type: "spring",
  stiffness: 100,
  damping: 10
}}

```

```
// Inertia (momentum-based)
transition={{
  type: "inertia",
  velocity: 20,
  power: 0.8
}}
```

Spring Animation: Feels natural and bouncy!

Chapter 5

Animating Between Conditional Values

5.1 Conditional Animation

Change animation based on state:

```
import { motion } from 'framer-motion';
import { useState } from 'react';

export default function Toggle() {
  const [isOn, setIsOn] = useState(false);

  return (
    <motion.div
      animate={{
        backgroundColor: isOn ? '#ff0000' : '#00ff00',
        scale: isOn ? 1.2 : 1
      }}
      transition={{ duration: 0.3 }}
    >
      <button onClick={() => setIsOn(!isOn)}>
        {isOn ? 'On' : 'Off'}
      </button>
    </motion.div>
  );
}
```

When state changes, animation automatically runs!

5.2 Complex Conditional Animation

```
export default function Menu({ isOpen }) {
  return (
    <motion.div
```

```
    animate={{
      x: isOpen ? 0 : -100,
      opacity: isOpen ? 1 : 0,
      pointerEvents: isOpen ? 'auto' : 'none'
    }}
    transition={{ duration: 0.4, type: 'spring' }}
  >
    <nav>Menu items</nav>
  </motion.div>
);
}
```

Power: Animation reacts to state changes automatically!

Chapter 6

Adding Entry Animations

6.1 Entry Animation Pattern

Animation when component first appears:

```
import { motion } from 'framer-motion';

export default function Card() {
  return (
    <motion.div
      initial={{ opacity: 0, scale: 0.9 }}
      animate={{ opacity: 1, scale: 1 }}
      transition={{ duration: 0.4 }}
    >
      <h2>Card Title</h2>
      <p>Card content</p>
    </motion.div>
  );
}
```

6.2 Different Entry Animations

6.2.1 Fade In

```
initial={{ opacity: 0 }}
animate={{ opacity: 1 }}
```

6.2.2 Slide In from Left

```
initial={{ x: -100, opacity: 0 }}
animate={{ x: 0, opacity: 1 }}
```

6.2.3 Scale Up

```
initial={{ scale: 0, opacity: 0 }}
animate={{ scale: 1, opacity: 1 }}
```

6.2.4 Rotate In

```
initial={{ rotate: -180, opacity: 0 }}
animate={{ rotate: 0, opacity: 1 }}
```

6.3 List Entry Animation

```
export default function List() {
  const items = ['Item 1', 'Item 2', 'Item 3'];

  return (
    <ul>
      {items.map((item, i) => (
        <motion.li
          key={i}
          initial={{ opacity: 0, x: -20 }}
          animate={{ opacity: 1, x: 0 }}
          transition={{ delay: i * 0.1 }}
        >
          {item}
        </motion.li>
      ))}
    </ul>
  );
}
```

Each item delays 0.1s more than previous!

Effect: Cascade animation as items appear!

Chapter 7

Animating Element Disappearances / Removal

7.1 Exit Animation with AnimatePresence

```
import { motion, AnimatePresence } from 'framer-motion';
import { useState } from 'react';

export default function Card() {
  const [isVisible, setIsVisible] = useState(true);

  return (
    <AnimatePresence>
      {isVisible && (
        <motion.div
          initial={{ opacity: 0, scale: 0.9 }}
          animate={{ opacity: 1, scale: 1 }}
          exit={{ opacity: 0, scale: 0.9 }}
          transition={{ duration: 0.3 }}
        >
          <p>Card content</p>
          <button onClick={() => setIsVisible(false)}>
            Close
          </button>
        </motion.div>
      )}
    </AnimatePresence>
  );
}
```

Key points:

- `AnimatePresence` wraps conditional rendering
- `exit` prop defines exit animation
- Component animates out before unmounting

7.2 List Item Removal

```
export default function TodoList() {
  const [todos, setTodos] = useState([]);

  return (
    <AnimatePresence>
      {todos.map(todo => (
        <motion.div
          key={todo.id}
          initial={{ opacity: 0, x: -20 }}
          animate={{ opacity: 1, x: 0 }}
          exit={{ opacity: 0, x: 20 }}
          transition={{ duration: 0.2 }}
        >
          <p>{todo.text}</p>
          <button onClick={() => deleteTodo(todo.id)}>
            Delete
          </button>
        </motion.div>
      ))}
    </AnimatePresence>
  );
}
```

Result: Smooth removal animation instead of instant disappear!

Chapter 8

Making Elements “Pop” With Hover Animations

8.1 Hover Animation

```
import { motion } from 'framer-motion';

export default function Button() {
  return (
    <motion.button
      whileHover={{ scale: 1.1 }}
      whileTap={{ scale: 0.95 }}
    >
      Click me!
    </motion.button>
  );
}
```

8.2 Hover Props

- `whileHover` - Animation while hovering
- `whileTap` - Animation while clicking
- `whileFocus` - Animation while focused

8.3 Complex Hover Animation

```
<motion.div
  whileHover={{
    scale: 1.1,
    boxShadow: '0 10px 30px rgba(0,0,0,0.3)',
    y: -5
  }}
  whileTap={{ scale: 0.95 }}
```

```
    transition={{ duration: 0.2 }}
  >
    
    <h3>Product Name</h3>
    <p>$99.99</p>
  </motion.div>
```

8.4 Icon Pop Effect

```
<motion.div
  whileHover={{ scale: 1.3, rotate: 10 }}
  whileTap={{ scale: 0.9 }}
  transition={{
    type: 'spring',
    stiffness: 400,
    damping: 10
  }}
>
  <LikeButton />
</motion.div>
```

Perfect for: Interactive buttons, product cards, icons!

Chapter 9

Reusing Animation States with Variants

9.1 What are Variants?

Predefined animation states you can reuse:

```
const boxVariants = {
  hidden: { opacity: 0, y: -20 },
  visible: { opacity: 1, y: 0 },
  hover: { scale: 1.1 }
};
```

```
export default function Box() {
  return (
    <motion.div
      variants={boxVariants}
      initial="hidden"
      animate="visible"
      whileHover="hover"
    >
      Content
    </motion.div>
  );
}
```

9.2 Reusable Variants

```
const itemVariants = {
  hidden: { opacity: 0, x: -50 },
  visible: { opacity: 1, x: 0 },
  exit: { opacity: 0, x: 50 }
};
```

```
export default function List() {
```

```
return (  
  <ul>  
    {items.map(item => (  
      <motion.li  
        key={item.id}  
        variants={itemVariants}  
        initial="hidden"  
        animate="visible"  
        exit="exit"  
      >  
        {item.text}  
      </motion.li>  
    ))}  
  </ul>  
);  
}
```

9.3 Variants with Transitions

```
const cardVariants = {  
  hidden: { opacity: 0, scale: 0.8 },  
  visible: {  
    opacity: 1,  
    scale: 1,  
    transition: { duration: 0.4 }  
  },  
  hover: {  
    scale: 1.05,  
    transition: { duration: 0.2 }  
  }  
};
```

Benefit: Consistency and easy reuse across components!

Chapter 10

Nested Animations & Variants

10.1 Parent-Child Animation

```
const containerVariants = {
  hidden: { opacity: 0 },
  visible: {
    opacity: 1,
    transition: {
      staggerChildren: 0.1
    }
  }
};

const itemVariants = {
  hidden: { opacity: 0, y: -10 },
  visible: { opacity: 1, y: 0 }
};

export default function List() {
  return (
    <motion.ul
      variants={containerVariants}
      initial="hidden"
      animate="visible"
    >
      {items.map(item => (
        <motion.li key={item.id} variants={itemVariants}>
          {item.text}
        </motion.li>
      ))}
    </motion.ul>
  );
}
```

10.2 How Stagger Works

Container animates with `staggerChildren: 0.1`

Child 1: starts at 0ms

Child 2: starts at 100ms

Child 3: starts at 200ms

Child 4: starts at 300ms

10.3 Delaying Parent Animation

```
const containerVariants = {
  hidden: { opacity: 0 },
  visible: {
    opacity: 1,
    transition: {
      delayChildren: 0.3,
      staggerChildren: 0.1
    }
  }
};
```

`/* Wait 0.3s, then stagger children 0.1s apart */`

Effect: Cascading animations feel professional!

Chapter 11

Animating Staggered Lists

11.1 List Animation Pattern

```
const listVariants = {
  hidden: { opacity: 0 },
  visible: {
    opacity: 1,
    transition: {
      staggerChildren: 0.2,
      delayChildren: 0.1
    }
  }
}

const itemVariants = {
  hidden: { opacity: 0, x: -100 },
  visible: {
    opacity: 1,
    x: 0,
    transition: { type: 'spring', stiffness: 100 }
  }
}

export default function TodoList({ todos }) {
  return (
    <motion.ul
      variants={listVariants}
      initial="hidden"
      animate="visible"
    >
      {todos.map(todo => (
        <motion.li
          key={todo.id}
          variants={itemVariants}
        >
```

```
        {todo.title}
      </motion.li>
    )}
  </motion.ul>
);
}
```

11.2 Different Stagger Directions

```
// Down-up (default)
staggerChildren: 0.1

// Back-to-front
delayChildren: (children.length - index) * 0.05

// Custom order
transition: {
  staggerChildren: 0.1,
  delayChildren: index * 0.05
}
```

Result: Polished cascading list animations!

Chapter 12

Animating Colors & Working with Keyframes

12.1 Color Animation

```
import { motion } from 'framer-motion';

export default function ColorBox() {
  return (
    <motion.div
      animate={{
        backgroundColor: ['#ff0000', '#00ff00', '#0000ff']
      }}
      transition={{
        duration: 3,
        repeat: Infinity
      }}
      style={{ width: 100, height: 100 }}
    />
  );
}
```

12.2 Keyframes Array

```
animate={{
  // Animate through multiple values
  x: [0, 100, 100, 0],
  opacity: [1, 0.5, 0.5, 1]
}}
transition={{
  duration: 2,
  repeat: Infinity
}}
```

Goes through values sequentially!

12.3 Timing for Each Keyframe

```
animate={{
  backgroundColor: ['#ff0000', '#00ff00', '#0000ff']
}}
transition={{
  duration: 3,
  times: [0, 0.5, 1], // When each color shows
  repeat: Infinity
}}

/* Red for first 1.5s, Green for 0.75s, Blue for 0.75s */
```

12.4 Complex Color Animation

```
<motion.div
  animate={{
    background: [
      'linear-gradient(to right, red, blue)',
      'linear-gradient(to right, blue, green)',
      'linear-gradient(to right, green, red)'
    ]
  }}
  transition={{
    duration: 4,
    repeat: Infinity
  }}
/>
```

Smooth: Color transitions feel fluid!

Chapter 13

Imperative Animations

13.1 Using useAnimation Hook

Programmatic control over animations:

```
import { motion, useAnimation } from 'framer-motion';
import { useEffect } from 'react';

export default function Box() {
  const controls = useAnimation();

  useEffect(() => {
    controls.start({
      x: 100,
      transition: { duration: 0.5 }
    });
  }, [controls]);

  return (
    <motion.div
      animate={controls}
      initial={{ x: 0 }}
    >
      Box
    </motion.div>
  );
}
```

13.2 Button Triggered Animation

```
export default function AnimatedBox() {
  const controls = useAnimation();

  const handleClick = async () => {
    await controls.start({ x: 100 });
  };
}
```

```
    await controls.start({ y: 100 });
    await controls.start({ x: 0, y: 0 });
  };

  return (
    <div>
      <motion.div animate={controls} />
      <button onClick={handleClick}>Animate</button>
    </div>
  );
}
```

13.3 Sequence of Animations

```
const animate = async () => {
  await controls.start({ opacity: 0.5 });
  await controls.start({ scale: 1.2 });
  await controls.start({ rotate: 360 });
  await controls.start({ opacity: 1 });
};

// Each animation waits for previous to finish
```

Power: Full control over animation timing and sequences!

Chapter 14

Animating Layout Changes

14.1 layoutId for Shared Layout Animation

```
import { motion, AnimateSharedLayout } from 'framer-motion';
import { useState } from 'react';

export default function Tabs() {
  const [active, setActive] = useState(0);

  return (
    <AnimateSharedLayout>
      <div>
        {[0, 1, 2].map(i => (
          <motion.button
            key={i}
            onClick={() => setActive(i)}
            layoutId={active === i ? 'active' : undefined}
          >
            Tab {i}
          </motion.button>
        ))}
      </div>
    </AnimateSharedLayout>
  );
}
```

14.2 Animated Flex Layout

```
<motion.div layout>
  {items.map(item => (
    <motion.div
      key={item.id}
      layout
      animate={{ opacity: isActive ? 1 : 0.5 }}
    >
```

```
      {item.name}  
    </motion.div>  
  )}  
</motion.div>
```

Add layout prop to animate layout changes!

Effect: Smooth transitions as items resize/move!

Chapter 15

Orchestrating Multi-Element Animations

15.1 Coordinating Multiple Elements

```
export default function MultiAnimation() {
  const containerControls = useAnimation();
  const textControls = useAnimation();
  const buttonControls = useAnimation();

  const handleStart = async () => {
    // Run in parallel
    await Promise.all([
      containerControls.start({ scale: 1.05 }),
      textControls.start({ opacity: 1 }),
      buttonControls.start({ y: 0 })
    ]);
  };

  return (
    <div>
      <motion.div animate={containerControls}>
        <motion.p animate={textControls}>Text</motion.p>
        <motion.button animate={buttonControls}>
          Click
        </motion.button>
      </motion.div>
    </div>
  );
}
```

15.2 Sequential Multi-Element Animation

```
const animateSequence = async () => {
  // 1. Header fades in
```

```
await headerControls.start({ opacity: 1 });

// 2. Then content slides in
await contentControls.start({ x: 0 });

// 3. Then footer appears
await footerControls.start({ opacity: 1 });
};
```

15.3 Staggered Multi-Element

```
const animateAll = async () => {
  await Promise.all([
    element1Controls.start({ x: 100 }),
    element2Controls.start({ x: 100 }),
    element3Controls.start({ x: 100 })
  ]);
};
```

```
// All animate simultaneously
```

Control: Complex choreography of multiple animations!

Chapter 16

Combining Animations With Layout Animations

16.1 Layout + Motion Together

```
import { motion, LayoutGroup } from 'framer-motion';

export default function Filter() {
  const [selected, setSelected] = useState('all');

  return (
    <LayoutGroup>
      <div>
        {categories.map(cat => (
          <motion.button
            key={cat}
            onClick={() => setSelected(cat)}
            layout
            animate={{
              backgroundColor: selected === cat
                ? '#3498db'
                : '#ecf0f1'
            }}
          >
            {cat}
          </motion.button>
        ))}
      </div>

      {/* Content animates with layout change */}
      <motion.div layout>
        {filtered.map(item => (
          <motion.div key={item.id} layout>
            {item.name}
          </motion.div>
        ))}
      </motion.div>
    </LayoutGroup>
  );
}
```

```
    ))}  
  </motion.div>  
</LayoutGroup>  
);  
}
```

Result: Smooth layout shifts with coordinated animations!

Chapter 17

Animating Shared Elements

17.1 Shared Element Transition

Animate same element across pages:

```
// Product List
<motion.div
  layoutId={`product-${id}`}
  onClick={() => navigate(`/product/${id}`)}
  animate={{ scale: 1 }}
  whileHover={{ scale: 1.05 }}
>
  <img src={image} />
  <h3>{name}</h3>
</motion.div>

// Product Detail Page
<motion.div layoutId={`product-${id}`}>
  <img src={image} />
  <h3>{name}</h3>
  <p>{description}</p>
</motion.div>
```

Same layoutId = animation between pages!

Magic: Seamless element transitions between screens!

Chapter 18

Re-triggering Animations via Keys

18.1 Using Keys to Reset Animations

```
export default function Card({ cardId }) {
  return (
    <motion.div
      key={cardId} // Change key = restart animation
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      transition={{ duration: 0.5 }}
    >
      Card {cardId}
    </motion.div>
  );
}

// When cardId changes, component remounts with animation
```

18.2 Forcing Re-trigger

```
const [trigger, setTrigger] = useState(0);

const handleReanimate = () => {
  setTrigger(prev => prev + 1);
};

return (
  <motion.div
    key={trigger}
    initial={{ scale: 0 }}
    animate={{ scale: 1 }}
  >
    Content
  </motion.div>
);
```

Trick: Key change = re-trigger initial animation!

Chapter 19

Scroll-based Animations

19.1 Using useViewportScroll

```
import { motion, useViewportScroll, useTransform } from 'framer-motion';

export default function ScrollAnimation() {
  const { scrollY } = useViewportScroll();

  // Transform scroll position to rotation
  const rotate = useTransform(
    scrollY,
    [0, 500],      // Input range
    [0, 360]       // Output range
  );

  return (
    <motion.div style={{ rotate }}>
      Rotates as you scroll!
    </motion.div>
  );
}
```

19.2 Scroll-based Opacity

```
const opacity = useTransform(
  scrollY,
  [0, 300],        // When at 0-300px
  [0, 1]           // Fade 0 to 1
);

return <motion.div style={{ opacity }} />;
```

19.3 Multiple Scroll Transforms

```
const rotate = useTransform(scrollY, [0, 500], [0, 360]);
```

```
const scale = useTransform(scrollY, [0, 500], [1, 2]);
const x = useTransform(scrollY, [0, 500], [0, 100]);

return (
  <motion.div style={{ rotate, scale, x }}>
    Multiple transforms!
  </motion.div>
);
```

Impressive: Scroll-driven animations without listeners!

Quick Summary

Animation Methods

1. **CSS Transitions** - Simple state changes
2. **CSS Animations** - Complex keyframes
3. **Framer Motion** - Powerful, flexible animations

Framer Motion Essentials

- **initial** - Starting state
- **animate** - Target state
- **exit** - Exit state (with `AnimatePresence`)
- **transition** - How to animate
- **variants** - Reusable animations

Key Props

- **whileHover** - Hover animation
- **whileTap** - Click animation
- **layout** - Layout animations
- **layoutId** - Shared elements

Best Practices

1. Keep animations under 500ms for UI
2. Use spring animations for natural feel
3. Combine enter/exit animations smoothly
4. Use variants for consistency
5. Always wrap conditional renders in `AnimatePresence`
6. Test performance on mobile devices

Animation Checklist

- Smooth transitions between states
- Loading states feel responsive
- Hover effects are satisfying
- Exit animations are smooth
- No layout shift surprises
- 60fps on all devices

Common Mistakes to Avoid

- Animations too slow (confuse users)
- Animations too fast (feel janky)
- Conditional content without `AnimatePresence`
- Over-animating (feels gimmicky)
- Forgetting exit animations
- Ignoring mobile performance

Smooth animations make apps feel polished and delightful!